Axial: A 3D Strategy Game with Gesture-Based Hand Tracking Controls

Caden Calderon

December 11, 2025

1 Introduction

1.1 Game Overview

Axial is a three-dimensional strategy game that reimagines the classic Connect Four as a spatial challenge. Two players take turns dropping colored pieces into a transparent 3D grid, where gravity pulls each piece down to the lowest available position in its column. The first player to align four of their pieces in a row wins.

What makes Axial unique is the playing field: instead of a flat 2D board, pieces exist in a $6 \times 6 \times 7$ volumetric grid containing 252 cells. Players must think in three dimensions, watching for winning lines that can form horizontally, vertically, diagonally, or through any of the 3D space diagonals. In total, there are 13 different directions a winning line can take.

1.2 Genre and Platform

- **Genre:** 3D Gravity-Based Strategy
- **Players:** Single-player versus AI opponent
- **Platform:** Desktop PC with standard webcam
- **Input Method:** Hand tracking gestures (no keyboard or mouse required)
- **Visual Style:** Minimalist aesthetic with glowing pieces and a transparent grid structure

1.3 The Hand Tracking Experience

Rather than clicking with a mouse, players interact with Axial using their hands in front of a webcam. The game tracks hand position and recognizes a pinch gesture (thumb and index finger touching) to grab, move, and drop game pieces. This creates a more tactile, intuitive experience where players feel like they are physically manipulating pieces in 3D space. Players can also rotate the entire game board by using both hands: one hand holds the board while the other hand's pinch-and-drag motion spins it. This allows players to examine the board from any angle to spot threats and opportunities hidden from the default view.

2 How to Play

This section explains everything a player needs to know to start playing Axial.

2.1 Setup and Starting a Game

1. Ensure a webcam is connected and positioned to see your hands clearly
2. Launch the Axial application
3. Position yourself so your hands are visible in the camera frame (approximately arm's length from camera works well)
4. Select a difficulty level from the menu:
   - **Easy:** Forgiving AI, good for learning
   - **Medium:** Balanced challenge
   - **Hard:** Strong AI, requires careful strategy
   - **Nightmare:** Expert-level AI
5. Press "Start Game" to begin

2.2 Controls

All interaction happens through hand gestures captured by the webcam:

2.2.1 Grabbing a Piece

When it is your turn, a game piece spawns at a designated location. To pick it up:

1. Move your hand near the piece. When you are close enough, the piece will **glow** to indicate it is within grab range.
2. Perform a **pinch gesture** by touching your thumb tip to your index finger tip.
3. The piece is now attached to your hand.

2.2.2 Moving and Positioning

While holding a piece (maintaining the pinch):

- Move your hand **left/right** to position over different columns
- Move your hand **forward/back** (closer to or further from the camera) to reach different depth rows
- The piece **snaps** to column centers as you move, making precise placement easy
- A **ghost preview** shows where your piece will land in that column

2.2.3 Dropping a Piece

1. Position the piece over your desired column
2. Check the visual feedback:
   - **Green tint** = valid drop location
   - **Red tint** = column is full, choose another
3. **Release the pinch** (separate thumb and index finger)
4. The piece falls into position and your turn ends

2.2.4 Rotating the Board

To see the board from different angles:

1. With one hand, **pinch and hold** the game board (grab it like any object)
2. With your other hand, perform a **pinch gesture**
3. While both hands are pinching, **drag your second hand** to rotate:
   - Horizontal movement rotates the board left/right
   - Vertical movement tilts the board forward/back
4. Release either pinch to stop rotating

After each turn, the board automatically returns to its upright position so both players always start from the same view.

2.3 Visual Feedback Guide

The game provides visual cues to help players understand what is happening:

Table 1: Visual Feedback Elements

Visual Cue	**Meaning**
Orange piece	Human player's piece
Blue piece	AI opponent's piece
Glowing object	Your hand is close enough to grab this object
Green tint on held piece	Valid drop location selected
Red tint on held piece	Invalid location (column full)
Ghost/transparent piece	Preview of where your piece will land
Hand skeleton overlay	Shows the game is tracking your hand

2.4 Strategy Tips

1. **Control the center:** Pieces in the middle of the board connect to more potential winning lines
2. **Think in 3D:** Winning lines can go in 13 directions, so use board rotation to check all angles

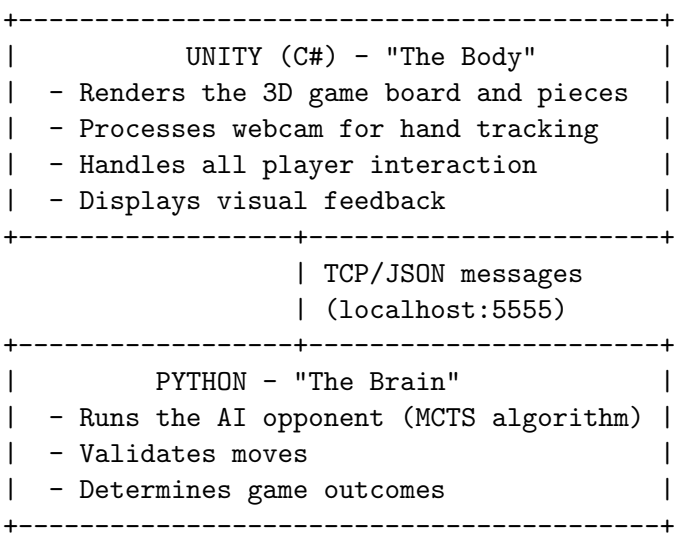
3. **Build forks:** Create positions where you threaten to win in two places at once, forcing the opponent into an impossible block
4. **Block early:** When the AI gets three in a row, block immediately before it becomes a forced win
5. **Watch the depth:** The most overlooked threats are the diagonal lines that go "into" the board

3 System Overview

Now that you understand how to play, this section provides a high-level view of how Axial works under the hood. We start broad and progressively go deeper in the following sections.

3.1 Architecture: Two Connected Systems

Axial is built as two separate programs that communicate with each other:

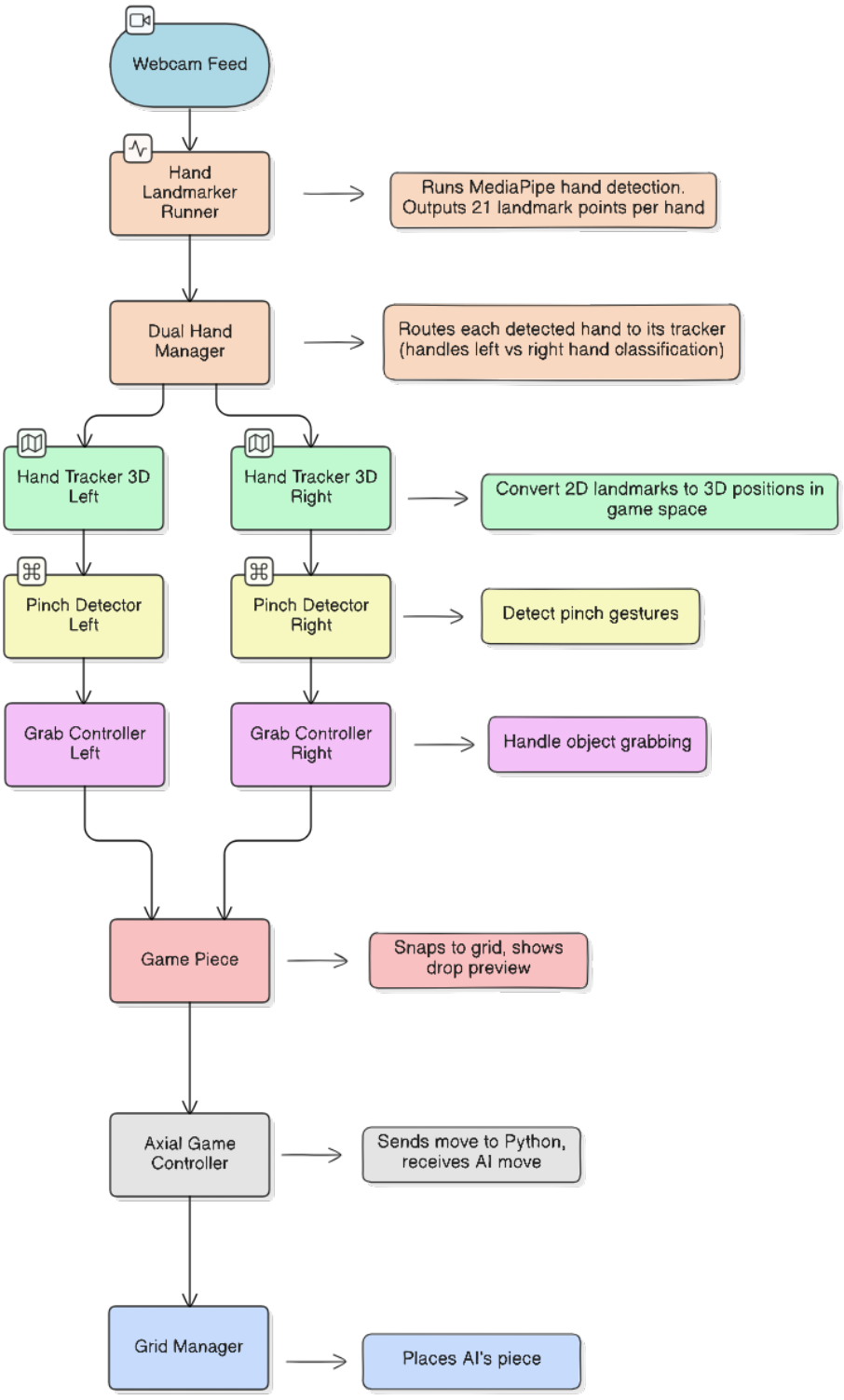


Why two systems? The AI requires heavy computation (thousands of game simulations per move). Running this in a separate Python process prevents the game from freezing or dropping frames while the AI "thinks." The Unity side stays smooth at 60 FPS while Python crunches numbers in the background.

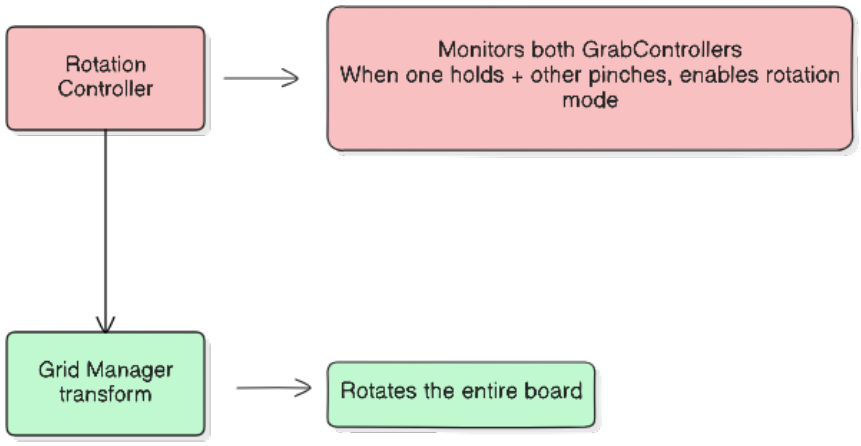
Communication happens through TCP sockets with JSON-formatted messages. When you drop a piece, Unity sends your move to Python; Python updates the game state, calculates the AI's response, and sends the AI's move back. This round-trip takes less than 2 milliseconds of network time, which is imperceptible.

3.2 Component Flow: How the Pieces Fit Together

Within the Unity application, several components work together to create the hand-tracking game experience. Here is how data flows from the webcam to a placed piece:



Additionally, when both hands are detected and one is holding an object:



This pipeline transforms raw webcam pixels into meaningful game actions. Each component has a single responsibility, making the system easier to debug and modify.

3.3 The 3D Game Board

Before diving into the technical details of hand tracking, it helps to understand what players are interacting with.

3.3.1 Board Structure

Imagine a transparent box divided into a grid of cells. The board has:

- **7 columns** along the X-axis (left to right)
- **6 rows** along the Z-axis (front to back, the "depth")
- **6 layers** along the Y-axis (bottom to top, where pieces stack)

This creates $7 \times 6 \times 6 = 252$ individual cells where pieces can rest. Each column is like a vertical tube: when you drop a piece into column (x, z), it falls to the lowest unoccupied Y position in that tube.

3.3.2 Cell Coordinates

Every cell has a coordinate (x, y, z) where:

- $x \in [0, 6]$: which column (left to right)
- $y \in [0, 5]$: which layer (bottom to top)
- $z \in [0, 5]$: which row (front to back)

The `GridManager` component tracks which player (if any) occupies each cell using a 3D array. It also handles converting between grid coordinates and actual 3D world positions for rendering and collision detection.

3.3.3 Winning Conditions

A player wins by placing four pieces in a consecutive line. In 3D, there are 13 possible directions for a winning line:

Table 2: The 13 Winning Directions

Category	Directions
Axial (3)	Along X, Along Y, Along Z
Flat diagonals (6)	XY diagonal, XZ diagonal, YZ diagonal (both orientations each)
Space diagonals (4)	Through the 3D corners (XYZ combinations)

After every piece placement, the game checks all 13 directions from the new piece's position to see if a winning line has formed.

4 Technical Challenge: Hand Tracking as Exclusive Input

The core technical challenge of this project was making hand tracking the **only** input method. No keyboard, no mouse, just hands in front of a webcam. This section explains the problems encountered and how they were solved.

4.1 Challenge Requirements

1. Players must grab, move, and place pieces using only hand gestures
2. The system must work with any standard webcam (no special hardware)
3. Interaction must feel responsive and natural, not laggy or jittery
4. The system must handle different hand sizes, skin tones, and gesture styles

4.2 The Depth Problem

The biggest challenge was determining **how far the hand is from the camera** (the Z-axis depth).

Why is this hard? A standard webcam produces a flat 2D image. It knows where your hand is horizontally (left/right) and vertically (up/down), but it cannot directly measure depth (near/far). However, players need depth control to reach different rows of the 3D board.

The solution: Infer depth from **apparent hand size**. When your hand is close to the camera, it appears large in the image. When far away, it appears small. By measuring how "big" the hand looks, we can estimate its distance.

4.3 Finding the Right Measurement

The concept sounds simple, but implementation required extensive experimentation. The challenge: how do you measure "hand size" in a way that stays consistent regardless of what gesture the player is making?

Eight different measurement approaches were tested:

Table 3: Hand Size Methods Evaluated

Method	Result	Problem
Index to Pinky distance	Failed	Rotated hands gave wrong readings
Wrist to Fingertip distance	Failed	Making a fist "teleported" the hand far away
Bounding box around hand	Partial	Fist shrank the box, moving depth
MediaPipe's built-in Z	Failed	Values were not calibrated for real distance
Palm width (X-axis only)	Partial	90-degree rotation broke it completely
Average of multiple measurements	Better	Still affected by some gestures
Palm skeleton only	Good	Minor drift when rotating
PalmMaxDimension	**Success**	Requires initial calibration

4.3.1 The Winning Approach: PalmMaxDimension

The key insight was twofold:

1. **Use only stable landmarks:** The wrist and four finger base knuckles do not move when you curl your fingers or make gestures. Measuring only these five points makes the system immune to gestures like fists or pointing.
2. **Take the maximum of three measurements:** Calculate palm width horizontally (X), palm width vertically (Y), and palm height. Whichever measurement is largest "wins" and drives the depth calculation. This handles rotation: when the palm faces the camera, X-width is large; when the hand rotates sideways, height becomes large instead. The maximum stays consistent.

4.4 Eliminating Jitter

Even with the correct measurement approach, hand tracking data is inherently noisy. MediaPipe (the hand detection library) produces slightly different coordinates each frame, even for a perfectly still hand. Without smoothing, objects would visibly shake.

Multiple smoothing layers were implemented throughout the pipeline:

1. **Hand size smoothing:** Each new measurement is blended with the previous value (92% old, 8% new), creating a stable rolling average.
2. **Dead zone:** Changes smaller than 2% are ignored completely. This prevents micro-jitter from accumulating.
3. **Z-position smoothing:** An additional smoothing pass specifically for depth, which is the noisiest axis.
4. **Grabbed object smoothing:** When holding an object, the object follows the hand with interpolation rather than snapping directly. This hides remaining jitter.

5. **Movement dead zone:** While holding an object, movements smaller than 5mm are ignored.

The result is smooth, stable interaction where players do not notice any jitter, while still feeling responsive to intentional movements.

4.5 Pinch Detection Reliability

The pinch gesture (thumb touching index finger) triggers grabbing and releasing. Making this reliable required two techniques:

Hysteresis thresholds: Instead of a single threshold ("pinching if distance less than X"), two thresholds are used:

- Pinch **starts** when finger distance drops below 0.8
- Pinch **ends** when finger distance rises above 1.2

The gap prevents rapid toggling when the hand is near the boundary. Without this, small tremors would cause objects to be grabbed and released repeatedly.

Confirmation frames: A pinch must be detected for 2 consecutive frames before triggering. This filters out single-frame detection errors.

4.6 Challenges Overcome Summary

1. **Depth from 2D camera:** Solved with PalmMaxDimension algorithm
2. **Gesture interference:** Solved by using only stable palm landmarks
3. **Rotation sensitivity:** Solved by taking maximum of three orthogonal measurements
4. **Tracking jitter:** Solved with multi-layer smoothing pipeline
5. **Pinch false triggers:** Solved with hysteresis and confirmation frames
6. **Object shake when grabbed:** Solved with smoothed following and movement dead zones

5 Technical Implementation Details

This section provides deeper technical details for readers interested in the implementation specifics.

5.1 Hand Tracking Pipeline

5.1.1 MediaPipe Integration

Hand detection uses Google's MediaPipe Hand Landmarker, which processes each webcam frame and outputs 21 landmark points per detected hand:

```
Landmark indices:
    0: Wrist
  1-4: Thumb (base to tip)
  5-8: Index finger (base to tip)
 9-12: Middle finger (base to tip)
13-16: Ring finger (base to tip)
17-20: Pinky (base to tip)
```

Each landmark has normalized (x, y) coordinates in the range $[0, 1]$, representing percentage of frame width/height. MediaPipe also provides a relative z value per landmark, but this represents finger depth relative to the palm, not absolute distance from camera.

5.1.2 Dual Hand Support

The `DualHandManager` component routes detected hands to separate left/right trackers. MediaPipe provides handedness classification, but from the camera's perspective (mirrored). The manager corrects this:

Listing 1: Handedness Correction

```
// MediaPipe "Left" = camera sees left side = user's RIGHT hand
if (categoryName.Contains("left"))
    return Handedness.Right;
else if (categoryName.Contains("right"))
    return Handedness.Left;
```

A confidence threshold (0.5) filters uncertain classifications to prevent hands from swapping erratically.

5.1.3 3D Position Calculation

`HandTracker3D` converts 2D normalized coordinates to 3D world positions:

- **X position:** Normalized X mapped to a configurable world width
- **Y position:** Normalized Y mapped to a configurable world height (inverted, since screen Y increases downward)
- **Z position:** Calculated from hand size using PalmMaxDimension algorithm

The PalmMaxDimension calculation:

Listing 2: PalmMaxDimension Algorithm

```
// Measure three dimensions of the palm
float widthX = Abs(indexBase.x - pinkyBase.x);  // Horizontal
float widthY = Abs(indexBase.y - pinkyBase.y);  // Vertical
float height = Distance(wrist, middleBase);      // Palm height

// Normalize each to 0-1 based on calibrated min/max values
float normX = InverseLerp(minX, maxX, widthX);
float normY = InverseLerp(minY, maxY, widthY);
float normH = InverseLerp(minH, maxH, height);

// Winner takes all - largest dimension drives depth
float handSize = Max(normX, normY, normH);

// Map to Z range
float zPosition = Lerp(minZ, maxZ, handSize);
```

5.1.4 Thread Safety

MediaPipe may invoke callbacks from background threads, but Unity requires all API calls on the main thread. A thread-safe caching pattern handles this:

Listing 3: Thread-Safe Data Transfer

```
// Background thread (MediaPipe callback)
void OnHandDataReceived(HandLandmarkerResult result) {
    lock (_lock) {
        result.CloneTo(ref _cachedResult);
        _hasNewData = true;
    }
}

// Main thread (Unity Update loop)
void Update() {
    if (_hasNewData) {
        lock (_lock) {
            ProcessHandData(_cachedResult);
            _hasNewData = false;
        }
    }
}
```

5.2 Grab System

The `GrabController` manages object interaction with extensive smoothing to ensure stability.

5.2.1 Proximity Detection

Every 50 milliseconds, the controller checks for nearby grabbable objects:

1. Sphere-cast from the smoothed pinch point position
2. Find all objects with `GrabbableObject` component within grab radius
3. Select the nearest valid object (not already grabbed, grabbing enabled)
4. Call `OnNearby()` on the nearest object to trigger glow effect
5. Call `OnNotNearby()` on the previous nearest if it changed

This provides visual feedback before the player pinches, so they know what they will grab.

5.2.2 Grab Smoothing Pipeline

When an object is grabbed, multiple smoothing stages prevent jitter:

Listing 4: Smoothed Object Following

```
// 1. Smooth the raw pinch point
_smoothedPinchPoint = Lerp(rawPinchPoint, _smoothedPinchPoint, 0.7f);

// 2. Calculate target position
Vector3 target = _smoothedPinchPoint + grabOffset;
```

```
6
7  // 3. Apply movement dead zone
8  if (Distance(target, currentPos) < 0.005f)
9      return;  // Ignore tiny movements
10
11 // 4. Smoothly interpolate toward target
12 newPosition = Lerp(currentPos, target, followSpeed * deltaTime);
```

Additionally, grabbed objects are forced into kinematic mode, disabling physics interference that could cause unexpected movement.

5.3 Grid Management

The `GridManager` handles board state and piece placement.

5.3.1 Board Data Structure

Listing 5: Board Representation

```
1  // 3D array: [column, layer, row]
2  private int[,,] boardGrid = new int[7, 6, 6];
3  // Values: 0 = empty, 1 = human player, 2 = AI player
```

5.3.2 World Position Mapping

Converting grid coordinates to world positions:

Listing 6: Grid to World Conversion

```
1  public Vector3 GetCellCenterWorld(int x, int y, int z)
2  {
3      // Calculate position in grid's local space
4      float halfCell = cellSpacing / 2f;
5      Vector3 localPos = new Vector3(
6          x * cellSpacing + halfCell,
7          y * cellSpacing + halfCell,
8          z * cellSpacing + halfCell
9      );
10     // Transform to world space (accounts for grid rotation/position)
11     return transform.TransformPoint(localPos);
12 }
```

This approach allows the entire board to be rotated while maintaining correct cell positions.

5.3.3 Piece Placement Flow

When a piece is dropped:

1. **Validate:** Check if column (x, z) has space
2. **Find target:** Get lowest empty Y in that column
3. **Mark immediately:** Set `boardGrid[x, y, z] = playerNumber` before animation (prevents race conditions if AI responds quickly)

4. **Parent piece:** Attach to grid transform so piece inherits board rotation
5. **Animate fall:** Move piece downward using `Vector3.MoveTowards`
6. **Snap to final:** Set exact position when animation completes
7. **Lock piece:** Disable grabbing on the placed piece
8. **Check win:** Scan all 13 directions from new piece
9. **Fire event:** Notify game controller that placement is complete

5.4 Two-Hand Rotation

The `RotationController` enables board rotation using both hands.

5.4.1 Activation Conditions

Rotation activates when:

- One hand is holding an object (the board)
- The other hand performs a pinch while not holding anything

Listing 7: Rotation Trigger Logic

```
if (leftGrabController.IsHoldingObject) {
    if (rightPinchDetector.IsPinching &&
        !rightGrabController.IsHoldingObject) {
        StartRotation(leftGrabController, rightPinchDetector);
    }
}
// (Same check with hands reversed)
```

5.4.2 Rotation Calculation

The rotating hand's index finger tip position is tracked. Movement from the previous frame determines rotation:

- Horizontal (X) movement → Yaw rotation (spin left/right)
- Vertical (Y) movement → Pitch rotation (tilt forward/back)

Rotation uses `RotateAround()` with the grid's calculated center point as the pivot, ensuring the board spins around its middle rather than a corner.

5.4.3 Automatic Reset

After each piece lands, the grid smoothly animates back to upright:

Listing 8: Grid Reset Animation

```
void UpdateGridReset() {
    // Rotate toward upright at 180 degrees per second
    gridTransform.rotation = Quaternion.RotateTowards(
        gridTransform.rotation,
        uprightRotation,
```

```
6           180f * Time.deltaTime
7       );
8   }
```

Only the Y-rotation (yaw) resets; any tilt the player applied is preserved. If the player starts rotating manually, the automatic reset cancels.

5.5 Skeleton Visualization

A skeleton overlay helps players verify their hand is being tracked correctly. Twenty-five cylinder primitives connect adjacent landmarks:

- Wrist to each finger base (5 bones)
- Each finger: base to tip through joints (4 fingers $\times$ 3 segments + thumb $\times$ 3 = 15 bones + 5 wrist = 20 total finger bones)

Each frame, bones are repositioned, rotated, and scaled to match current landmark positions. Performance optimizations include cached transforms, disabled shadows, and no colliders.

6 AI Opponent

The AI opponent runs in the Python backend and uses Monte Carlo Tree Search (MCTS) to select moves. While the full algorithmic details are covered in a separate AI-focused report, key points relevant to gameplay include:

- **Thinking process:** The AI simulates thousands of random game continuations to estimate which move leads to the best outcome
- **Threat awareness:** The AI always takes immediate winning moves and blocks opponent wins
- **Difficulty scaling:** Higher difficulty means more simulations, leading to deeper analysis and stronger play
- **Response time:** Easy mode responds in under 1 second; Nightmare mode may take 2-3 seconds

Table 4: Difficulty Levels

Level	AI Simulations	Approximate Strength
Easy	200	Beatable with basic strategy
Medium	500	Competitive for casual players
Hard	1,000	Requires careful planning
Nightmare	2,000	Expert-level play

7 Evaluation

7.1 Counter Group Feedback

The game was tested by a counter group who provided feedback:

Positive:

- Overall impression was "great"
- Hand tracking controls were engaging and novel
- Visual design was clean and easy to read

Constructive:

- AI was considered too difficult for casual players
- Suggested reducing board size to make the game easier

7.2 Response to Feedback

Rather than reducing the board (which would change the fundamental game), I addressed the difficulty concern by implementing the four-tier difficulty system. Lower difficulties use fewer AI simulations, causing it to occasionally miss optimal moves while still playing competently. This preserves the full 3D experience while making the game accessible to players of all skill levels.

8 Member Responsibilities

This project was just me:

- Game design and rules
- Unity application (C#): visualization, UI, game logic
- Hand tracking system: MediaPipe integration, depth inference, gesture detection
- Grab and rotation systems with smoothing
- Grid management and piece physics
- Python backend: AI opponent using MCTS
- Testing and debugging
- Documentation

Asset Attribution: All visual assets (3D models, materials, UI elements) were created by me. No external assets were downloaded or purchased.

9 Conclusion

Axial demonstrates that hand tracking can serve as a viable exclusive input method for strategy games using only a standard webcam. The project successfully addressed the core technical challenges:

1. **Depth inference:** The PalmMaxDimension algorithm provides stable Z-axis positioning by combining three orthogonal palm measurements
2. **Gesture immunity:** Using only stable palm landmarks prevents gestures from affecting position

3. **Jitter elimination:** Multi-layer smoothing throughout the pipeline ensures stable, responsive interaction
4. **Two-hand interaction:** Simultaneous tracking enables intuitive board rotation for 3D exploration
5. **Visual feedback:** Proximity highlighting and drop previews make the interface predictable

The game extends classic Connect Four into three dimensions while the hand-tracking controls make the spatial gameplay feel tangible and immersive.

9.1 Future Improvements

- **Per-user calibration:** Auto-calibrate depth ranges based on individual hand sizes
- **WebGL deployment:** Enable browser-based play without installation
- **Networked multiplayer:** Human vs. human matches over the internet
- **Additional gestures:** Explore gestures for menu navigation and undo functionality